Configuration

Bu loyihaning chizmasi. model qanday quriladi, va qanday o'qitiladi hamasi kod ichida emas alohida .yaml filelda yoziladi. kod shu filelni o'qiydi.

Masalan: Qurilishda quruvchilar g'ilt terishni, elevator motorlarini tom yopishni biladi. va arxitektorning chizmasiga qarab qanday qilishni ko'rib bajaradi.

Config - Arxitektar chizmasi.
kod - quruvchilar

Loyiha root da configs/name.yaml filening ichiga: model design yoziladi.

vocab

① vocab_size - lug'at hajmi.
=> Model taniydigan tokenlar soni.
=> tokenizerdagi lug'at hajmi bilan bir xil bo'lishi kerak. Aks holda yo'q ID bn xatolik beradi.

② block_size - context window.
=> model bir vaqtda ko'ra oladigan tokenlar soni.
=> katta blocksize - model uzunroq kontekstni tushunadi lekin sekinroq va ko'p xotira talab qiladi.

③ n_embd – Har bir token vektorining uzunligi.

-> Model o'rganadigan maydonning kengligi. Vektor qancha uzun bo'lsa shuncha ko'p ma'lumot sig'adi. Trade-off – dimension katta bo'lsa compute og'irlashadi va xotira ham ko'p sarflanadi.

④ n_layer – model chuqurligi.

=> Transformer bloklar soni.

-> Model ma'lumotni nechta bosqichda qayta ishlashi. Layer 1 (basic grammar, so'zlar ketma-ketligi). Layer 2 (so'zlarni birlashtirish, qisqa frazalar).

=> Keyingi layerlar – oldingi natijalarni o'rganadi yuqoriroq – murakkabroq.

⑤ n_head – attention headlar soni.

=> Har bir layerdagi parallel attention heads

Head o'lchami => |n_embd / n_head|.

oddiy bo'lmaydigan kasr.

n_embd = 384.
n_head = 6.

head_dim = $\frac{384}{6} = 64$

(n_embd = 384)

Token vector [64 floats | 64 floats | .... | 64 floats]

↓ Attention Head 1 | Attention Head 2. | Attention Head 6.

⑥ dropout = 0.1

=> overfitting oldini oladi.

=> Train paytida tasodifiy 0.1=10% giperparametrlarni nolga aylantiradi. Tahlil yubormaydi. SLM da ishlatish qiyin. Chunki data kam bo'lganda model uni yodlab olishi mumkin bo'ladi. LLM larda weight decay dan foydalanamiz.

weight decay => weightlarni nolga tortib turadi. weightlar haddan tashqari katta bo'lib ketmaydi.

⑦ tie_weights - weight tying.

Input embedding => token o'girilganda ishlatiladigan vector.

Output embedding => token bashorat qilinganda ishlatiladigan vector.

Ikkitasi ham bir xil lug'atdan foydalanadi.

tie_weights = True) qilish

uzbek => English lug'at bor. (Ite lug'at.

olma => apple.

english => uzbek uchun yana lug'at kerak emas.

apple => olma !

olmaning apple ligini bilamiz va bu apple => olma degani.

② ffn_mult

Bu FFN ichidagi kengaytirilgan qatlam hajmini belgilaydi.

ffn * n_embd = ?

=> Modelning bilimlar ombori.

=> ffn_mult qancha katta bolsa, bilimi kengroq boladi lekin xotira va hisoblash ham shuncha kop boladi.